

WEEKLY ASSESSMENT 1

Section A — Theory

Q1. Difference between `is` and `==`, with an example.

Ans: `==` compares values (are the contents equal), while `is` compares identity (are they the same object in memory).

Example: `a = [1,2,3]; b = [1,2,3] → a == b` is True (same contents) but `a is b` is False (different objects).

Q2. Classify list, tuple, set, dict, str, int, float, bool, frozenset as Mutable/Immutable; why immutability matters.

Ans: Mutable: list, set, dict. Immutable: tuple, str, int, float, bool, frozenset.

Immutability matters because immutable objects are safe to use as dict keys/set members and can be shared across functions without risk of accidental modification (no aliasing bugs).

Q3. Predict `print(0.1 + 0.2 == 0.3)` and explain why.

```
print(0.1 + 0.2 == 0.3)
print(0.1 + 0.2)
```

Output:

```
False
0.30000000000000004
```

Ans: Floats are stored in binary (IEEE-754), and 0.1 and 0.2 cannot be represented exactly in binary, so tiny rounding errors creep in.

This is not a Python bug — compare floats with a tolerance (e.g. `abs(a-b) < 1e-9`) instead of `==`.

Q4. 3 concrete situations where you'd choose a tuple over a list.

Ans: 1) Returning multiple values from a function (e.g. `return (status, message)`) — the grouping should not be modified by the caller.

2) Using coordinates or fixed records (like (latitude, longitude)) as dictionary keys — only immutable types are hashable.

3) Representing data that must stay constant, e.g. `days_of_week = ("Mon", ..., "Sun")` — prevents accidental edits elsewhere in the code.

Q5. Why dict lookup is $O(1)$ average while `in` on a list is $O(n)$; explain without and then with the word 'hash'.

Ans: Without 'hash': a list has no shortcut — Python checks every element one by one until it finds a match, so time grows with size.

With 'hash': a dict converts each key into a hash number that directly points to a bucket/slot, so Python jumps almost straight to the value instead of scanning.

Q6. Difference between `range()` and `numpy.arange()` — at least 3 differences, one involving floats.

Ans: 1) `range()` only accepts integers, while `np.arange()` accepts floats too (e.g. `np.arange(0, 1, 0.25)`).

2) `range()` returns a lightweight lazy object; `np.arange()` returns an actual NumPy array stored in memory.

3) range() supports basic Python iteration only; np.arange() supports vectorized math (e.g. np.arange(5) * 2 works directly, range(5)*2 does not).

Q7. Difference between `append(x)`, `extend(x)`, `insert(i, x)` — show effect of adding x = [4, 5] to a = [1, 2, 3].

```
a = [1, 2, 3]
x = [4, 5]

a.append(x)      #a -> [1, 2, 3, [4, 5]]      (adds x AS ONE element)
a = [1, 2, 3]
a.extend(x)      # a -> [1, 2, 3, 4, 5]      (adds EACH element of x)
a = [1, 2, 3]
a.insert(1, x)   # a -> [1, [4, 5], 2, 3]    (inserts x as one element at index 1)
```

Ans: append() adds its argument as a single element; extend() unpacks and adds each item individually; insert(i, x) is like append but places the single element at a chosen index.

Q8. What does `a, \*b, c = [10, 20, 30, 40, 50]` do? Values of a, b, c? What about [10, 20] and [10]?

```
a, *b, c = [10, 20, 30, 40, 50]
print(a, b, c)

a, *b, c = [10, 20]
print(a, b, c)

a, *b, c = [10]
print(a, b, c)
```

Output:
10 [20, 30, 40] 50
10 [] 20
ValueError: not enough values to unpack (expected at least 2, got 1)

Ans: `\*b` is star unpacking: a takes the first element, c takes the last, and b (a list) absorbs everything in between — even if that's empty.

With only one element there aren't enough values for both a and c, so Python raises a ValueError.

Q9. Difference between `/`, `//`, `%`, `\*\*` — output for each on (-7, 2).

```
print(-7 / 2)    # true division
print(-7 // 2)   # floor division
print(-7 % 2)    # modulus
print(-7 ** 2)   # exponent
```

Output:
-3.5
-4
1
-49

Ans: / gives an exact float. // floors (rounds down), so -3.5 floors to -4. % returns the remainder with the sign of the divisor, so it's positive here. ** binds tighter than unary minus, so -7**2 means -(7**2) = -49.

Q10. Why is NumPy significantly faster than Python lists for numerical operations? 3 reasons, one about memory layout.

Ans: 1) Memory layout: a NumPy array stores same-type elements in one contiguous block, giving better CPU cache locality — a Python list stores scattered pointers.

- 2) Vectorized C loops: NumPy runs pre-compiled C code instead of looping in the slower Python interpreter.
- 3) No per-element overhead: NumPy stores raw values, while a Python int/float object carries extra metadata.

Section B — Output Prediction

Q11. matrix1 vs matrix2 — why does the second line behave differently?

```
matrix1 = [[0] * 3 for _ in range(3)]
matrix2 = [[0] * 3] * 3
matrix1[0][0] = 99
matrix2[0][0] = 99
print(matrix1)
print(matrix2)
```

Output:

```
[[99, 0, 0], [0, 0, 0], [0, 0, 0]]
[[99, 0, 0], [99, 0, 0], [99, 0, 0]]
```

Ans: matrix1's comprehension creates 3 SEPARATE inner lists. matrix2's ``* 3`` repeats a REFERENCE to the same inner list 3 times, so editing row 0 appears to edit every row.

Q12. x/y and a/b behaviour — why do ``x = x + [4]`` and ``a += [4]`` behave differently?

```
x = [1, 2, 3]
y = x
x = x + [4]
print("x =", x)
print("y =", y)

a = [1, 2, 3]
b = a
a += [4]
print("a =", a)
print("b =", b)
```

Output:

```
x = [1, 2, 3, 4]
y = [1, 2, 3]
a = [1, 2, 3, 4]
b = [1, 2, 3, 4]
```

Ans: ``x + [4]`` builds a new list and rebinds x, so y is unaffected. ``a += [4]`` mutates the original list in place, so b (pointing to the same object) also changes.

Q13. Value of ``i`` after ``for i in range(5): pass``; value of ``j`` after ``for j in range(0): pass``.

```
for i in range(5):
    pass
print(i)

for j in range(0):
    pass
print(j)
```

Output:

4

NameError: name 'j' is not defined

Ans: The loop variable survives after the loop with its last assigned value (4). range(0) never runs its body, so j is never created, causing a NameError.

Q14. Predict output of each print; which lines error?

```
print("10" + "20")
print(10 + 20)
print("10" * 3)
print(10 * "3")
print("10" + 20)
print(int("10") + 20)
```

Output:

1020

30

101010

3333333333

TypeError: can only concatenate str (not "int") to str

30

Ans: String + string concatenates; int + int adds; string * n repeats the string. `"10" + 20` errors because Python won't auto-convert types for +.

Q15. Predict output; explain short-circuit evaluation in one line.

```
print(0 or 5)
print(5 or 0)
print(None or "hello")
print(0 and 5)
print(5 and 10)
print([] or "default")
print([0] or "default")
```

Output:

5

5

hello

0

10

default

[0]

Ans: ``or`` returns the first truthy operand; ``and`` returns the first falsy operand — evaluation stops as soon as the result is decided.

[0] is a non-empty list, so it is truthy — only an empty container is falsy, regardless of its contents.

Section C — Coding Practice

Q16. Swap two numbers without a third variable — 3 methods.

```
def swap_arith(a, b):
    a = a + b
    b = a - b
    a = a - b
```

```

    return a, b

def swap_xor(a, b):
    a = a ^ b
    b = a ^ b
    a = a ^ b
    return a, b

def swap_tuple(a, b):
    a, b = b, a # most Pythonic
    return a, b

print(swap_arith(5, 10))
print(swap_xor(5, 10))
print(swap_tuple(5, 10))

```

Output:

```

(10, 5)
(10, 5)
(10, 5)

```

Ans: Tuple unpacking (a, b = b, a) is the most Pythonic — one readable line, works on any data type, no overflow risk.

Q17. Swiggy Order Calculator.

```

price = float(input("Enter item price: "))
qty = int(input("Enter quantity: "))

subtotal = price * qty
gst = round(subtotal * 0.18, 2)
delivery = 40.0 if subtotal < 500 else 0.0
total = round(subtotal + gst + delivery, 2)

print(f"Subtotal: ₹{subtotal:.2f}")
print(f"GST (18%): ₹{gst:.2f}")
print(f"Delivery: ₹{delivery:.2f}")
print("-----")
print(f"Total: ₹{total:.2f}")

```

Output:

```

price=150, qty=3
Subtotal: ₹450.00
GST (18%): ₹81.00
Delivery: ₹40.00
-----
Total: ₹571.00

```

Q18. Temperature converter with graceful invalid-unit handling.

```

def convert_temp(value, unit):
    unit = unit.upper()
    if unit == "C":
        return round(value * 9/5 + 32, 2), "F"
    elif unit == "F":
        return round((value - 32) * 5/9, 2), "C"
    else:
        return None, None

value = float(input("Enter value: "))

```

```

unit = input("Enter unit (C/F): ")
result, new_unit = convert_temp(value, unit)
if result is None:
    print("Error: unit must be 'C' or 'F'")
else:
    print(f"{value}{unit.upper()} = {result}{new_unit}")

```

Output:

```

(100, 'C') -> 212.0 F
(212, 'F') -> 100.0 C
(100, 'X') -> Error: unit must be 'C' or 'F'

```

Q19. Sum of digits of a 4-digit number WITHOUT string conversion.

```

n = int(input("Enter a 4-digit number: "))
total = 0
temp = n
while temp > 0:
    total += temp % 10    # peel off last digit
    temp //= 10          # drop last digit
print("Sum of digits:", total)

```

Output:

```

Input: 1234 -> Sum of digits: 10

```

Q20. Smart input parser — detect int, float, or string.

```

def detect(value):
    try:
        return int(value), "int"
    except ValueError:
        pass
    try:
        return float(value), "float"
    except ValueError:
        pass
    return value, "str"

raw = input("Enter a value: ")
val, kind = detect(raw)
print(f"{kind} {val}")

```

Output:

```

"42" -> int 42
"3.14" -> float 3.14
"hello" -> str hello

```

Q21. Indian Income Tax Calculator (New Regime, FY 2024–25) — slab-wise.

```

income = float(input("Enter annual income: "))

slabs = [
    (300000, 0.00),
    (400000, 0.05),
    (500000, 0.10),
    (600000, 0.15),
    (700000, 0.20),
    (float("inf"), 0.30),
]

```

```

remaining = income
tax = 0
for slab_amount, rate in slabs:
    if remaining <= 0:
        break
    taxable = min(remaining, slab_amount)
    slab_tax = taxable * rate
    tax += slab_tax
    if slab_tax > 0:
        print(f"{taxable:.0f} @ {rate*100:.0f}% = ₹{slab_tax:.0f}")
    remaining -= taxable

print(f"Total Tax: ₹{tax:.0f}")

```

Output:

```

Income: 800000
400000 @ 5% = ₹20000
100000 @ 10% = ₹10000
Total Tax: ₹30000

```

Q22. Validate Indian PAN card format WITHOUT regex.

```

def validate_pan(pan):
    if len(pan) != 10:
        return "Invalid PAN – reason: length must be exactly 10 characters"
    for i in range(5):
        if not (pan[i].isalpha() and pan[i].isupper()):
            return f"Invalid PAN – reason: position {i+1} must be an uppercase letter"
    for i in range(5, 9):
        if not pan[i].isdigit():
            return f"Invalid PAN – reason: position {i+1} must be a digit"
    if not (pan[9].isalpha() and pan[9].isupper()):
        return "Invalid PAN – reason: position 10 must be an uppercase letter"
    return "Valid PAN"

pan = input("Enter PAN: ")
print(validate_pan(pan))

```

Output:

```

ABCDE1234F -> Valid PAN
ABCDE123F  -> Invalid PAN – reason: length must be exactly 10 characters
abcde1234f -> Invalid PAN – reason: position 1 must be an uppercase letter

```

Q23. Bengaluru BESCO Electricity Bill (LT-2 Residential).

```

units = float(input("Enter units consumed: "))

slabs = [(50, 4.10), (50, 5.55), (100, 7.10)]
remaining = units
energy_charge = 0
for slab_size, rate in slabs:
    if remaining <= 0:
        break
    used = min(remaining, slab_size)
    energy_charge += used * rate
    remaining -= used
if remaining > 0:
    energy_charge += remaining * 8.15

```

```

fixed_charge = 110
tax = round(0.06 * (energy_charge + fixed_charge), 2)
total = round(energy_charge + fixed_charge + tax, 2)

print(f"Energy Charge: ₹{energy_charge:.2f}")
print(f"Fixed Charge: ₹{fixed_charge:.2f}")
print(f"Govt Tax (6%): ₹{tax:.2f}")
print(f"Total Bill: ₹{total:.2f}")

```

Output:

```

Units: 250
Energy Charge: ₹1600.00
Fixed Charge: ₹110.00
Govt Tax (6%): ₹102.60
Total Bill: ₹1812.60

```

Q24. Cricket Strike Rate Classifier — handle balls = 0.

```

runs = int(input("Runs scored: "))
balls = int(input("Balls faced: "))

if balls == 0:
    print("Cannot compute strike rate – no balls faced")
else:
    sr = (runs / balls) * 100
    if sr > 150:
        category = "Explosive (T20 finisher)"
    elif sr > 100:
        category = "Aggressive (Top order T20)"
    elif sr >= 75:
        category = "Steady (ODI anchor)"
    else:
        category = "Defensive (Test match)"
    print(f"Strike Rate: {sr:.2f} -> {category}")

```

Output:

```

runs=80, balls=40 -> Strike Rate: 200.00 -> Explosive (T20 finisher)
balls=0 -> Cannot compute strike rate – no balls faced

```

Q25. BMI Calculator using Indian BMI categories.

```

weight = float(input("Weight (kg): "))
height_cm = float(input("Height (cm): "))
height_m = height_cm / 100
bmi = weight / (height_m ** 2)

if bmi < 18.5:
    category = "Underweight"
elif bmi < 23.0:
    category = "Normal"
elif bmi < 25.0:
    category = "Overweight"
elif bmi < 30.0:
    category = "Obese Class I"
else:
    category = "Obese Class II"

print(f"BMI: {bmi:.2f} -> {category}")

```

Output:

weight=70, height=170 -> BMI: 24.22 -> Overweight

Q26. Right triangle number pattern for N = 5.

```
N = 5
for i in range(1, N + 1):
    for j in range(1, i + 1):
        print(j, end=' ')
    print()
```

Output:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Q27. Number diamond for N = 5.

```
N = 5
def print_row(i):
    print(' ' * (N - i), end='')
    for j in range(1, i + 1):
        print(j, end=' ')
    for j in range(i - 1, 0, -1):
        print(j, end=' ')
    print()

for i in range(1, N + 1):
    print_row(i)
for i in range(N - 1, 0, -1):
    print_row(i)
```

Output:

```
1
1 2 1
1 2 3 2 1
1 2 3 4 3 2 1
1 2 3 4 5 4 3 2 1
1 2 3 4 3 2 1
1 2 3 2 1
1 2 1
1
```

Q28. Hollow square of size N (border only).

```
N = 5
for i in range(N):
    for j in range(N):
        if i == 0 or i == N - 1 or j == 0 or j == N - 1:
            print('*', end=' ')
        else:
            print(' ', end=' ')
    print()
```

Output:

```
* * * * *
*       *
*       *
```

```
*      *
* * * * *
```

Q29. First 5 rows of Pascal's Triangle (no math.comb).

```
N = 5
for i in range(N):
    row = []
    value = 1
    for k in range(i + 1):
        row.append(value)
        value = value * (i - k) // (k + 1)
    print(' '.join(str(x) for x in row))
```

Output:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

Q30. Character pattern up to Nth alphabet using chr()/ord().

```
N = 5
for i in range(1, N + 1):
    for j in range(i):
        print(chr(ord('A') + j), end=' ')
    print()
```

Output:

```
A
A B
A B C
A B C D
A B C D E
```

Q31. All Armstrong numbers between 1 and 9999.

```
armstrong_numbers = []
for num in range(1, 10000):
    digits = [int(d) for d in str(num)]
    power = len(digits)
    if sum(d ** power for d in digits) == num:
        armstrong_numbers.append(num)

print(", ".join(map(str, armstrong_numbers)))
print("Count:", len(armstrong_numbers))
```

Output:

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407, 1634, 8208, 9474
Count: 16
```

Q32. First N Fibonacci terms + their sum (no recursion).

```
N = int(input("Enter N: "))
series = []
a, b = 0, 1
for _ in range(N):
    series.append(a)
    a, b = b, a + b
```

```
print(series)
print("Sum:", sum(series))
```

Output:

```
N=0 -> [], Sum: 0
N=1 -> [0], Sum: 0
N=10 -> [0, 1, 1, 2, 3, 5, 8, 13, 21, 34], Sum: 88
```

Q33. Optimized prime check (up to $\sqrt{N}$) + sum of primes up to N.

```
def is_prime(n):
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True

N = int(input("Enter N: "))
total = 0
for n in range(2, N + 1):
    if is_prime(n):
        total += n
print("Sum of primes:", total)
```

Output:

```
N=20 -> Sum of primes: 77
```

Q34. Reverse a number WITHOUT strings; palindrome check (negatives never palindrome).

```
def reverse_num(n):
    n = abs(n)
    rev = 0
    while n > 0:
        rev = rev * 10 + n % 10
        n //= 10
    return rev

def is_palindrome(n):
    if n < 0:
        return False
    return n == reverse_num(n)

print(is_palindrome(121))
print(is_palindrome(123))
print(is_palindrome(-121))
```

Output:

```
True
False
False
```

Q35. HCF via Euclidean algorithm (while loop) + LCM from HCF x LCM = a x b.

```
def hcf_lcm(a, b):
    x, y = a, b
    while y:
```

```

        x, y = y, x % y
    hcf = x
    lcm = (a * b) // hcf
    return hcf, lcm

```

```
print(hcf_lcm(12, 18))
```

Output:
(6, 36)

Q36. String scanner — count chars/words/vowels/consonants/digits/spaces/specials in ONE pass.

```

def scan(sentence):
    chars = words = vowels = consonants = digits = spaces = special = 0
    in_word = False
    for ch in sentence:
        chars += 1
        if ch == ' ':
            spaces += 1
            in_word = False
        else:
            if not in_word:
                words += 1
                in_word = True
            if ch.isalpha():
                if ch.lower() in "aeiou":
                    vowels += 1
                else:
                    consonants += 1
            elif ch.isdigit():
                digits += 1
            else:
                special += 1
    return chars, words, vowels, consonants, digits, spaces, special

```

```
print(scan("Order came on time, food was hot, packaging 10/10!"))
```

Output:
chars=50, words=9, vowels=14, consonants=20, digits=4, spaces=8, special=4

Q37. Analyze 10 marks without sort()/sorted(): max, min, avg, count above avg, 2nd highest.

```

marks = list(map(int, input("Enter 10 marks: ").split()))

highest = lowest = marks[0]
for m in marks:
    if m > highest: highest = m
    if m < lowest: lowest = m
average = sum(marks) / len(marks)
above_avg = sum(1 for m in marks if m > average)

second_highest = None
for m in marks:
    if m != highest and (second_highest is None or m > second_highest):
        second_highest = m

print("Max:", highest, "Min:", lowest, "Avg:", average)
print("Above average:", above_avg, "2nd highest:", second_highest)

```

Output:

Input: 78 45 92 63 88 71 55 99 60 82
Max: 99 Min: 45 Avg: 73.3
Above average: 5 2nd highest: 92

Q38. Word Frequency Analyzer for Swiggy reviews (case-insensitive, dict-based).

```
review = input("Enter review: ")
words = review.lower().split()

freq = {}
for w in words:
    cleaned = ''.join(ch for ch in w if ch.isalnum())
    freq[cleaned] = freq.get(cleaned, 0) + 1

sorted_freq = sorted(freq.items(), key=lambda pair: pair[1], reverse=True)
for word, count in sorted_freq:
    print(word, ":", count)

top_word, top_count = sorted_freq[0]
print("Most frequent:", top_word, top_count)
```

Output:

food : 3
was : 3
great : 1
...
Most frequent: food 3

Q39. Indian States CRUD — menu-driven dict program.

```
states = {"Karnataka": "Bengaluru", "Maharashtra": "Mumbai",
          "Tamil Nadu": "Chennai", "Gujarat": "Gandhinagar",
          "Kerala": "Thiruvananthapuram"}

while True:
    print("1.Add 2.Update 3.Delete 4.Search 5.Display 6.Exit")
    choice = input("Choose an option: ")
    if choice == "1":
        s = input("State: "); c = input("Capital: "); states[s] = c
    elif choice == "2":
        s = input("State to update: ")
        if s in states: states[s] = input("New capital: ")
        else: print("Not found")
    elif choice == "3":
        s = input("State to delete: ")
        states.pop(s, None)
    elif choice == "4":
        s = input("State to search: ")
        print(states.get(s, "Not found"))
    elif choice == "5":
        for s in sorted(states):
            print(s, "->", states[s])
    elif choice == "6":
        break
    else:
        print("Invalid option")
```

Output:

```
Search 'Kerala' -> Thiruvananthapuram
Search 'Assam' -> Not found
```

Q40. Set operations on Section A / Section B language sets.

```
A = {"Python", "Java", "C++", "SQL"}
B = {"Java", "JavaScript", "Python", "Go", "SQL"}

print("Both:", A & B)
print("Only A:", A - B)
print("Only B:", B - A)
print("Total unique:", A | B, "count:", len(A | B))
print("Exclusive to one:", A ^ B)
```

Output:

```
Both: {'Python', 'SQL', 'Java'}
Only A: {'C++'}
Only B: {'Go', 'JavaScript'}
Total unique count: 6
Exclusive to one: {'C++', 'Go', 'JavaScript'}
```

Q41. TCS stock prices tuple — max/min day, average, biggest gain, volatility.

```
prices = (3845, 3902, 3878, 3955, 4012, 3998, 4055)
days = ("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun")

highest, lowest = max(prices), min(prices)
high_day, low_day = days[prices.index(highest)], days[prices.index(lowest)]
average = sum(prices) / len(prices)

daily_changes = [prices[i+1] - prices[i] for i in range(len(prices) - 1)]
biggest_gain = max(daily_changes)
gain_idx = daily_changes.index(biggest_gain)
volatility = sum(abs(c) for c in daily_changes)

print(f"Max: {highest} ({high_day}) Min: {lowest} ({low_day})")
print(f"Average: {average:.2f}")
print(f"Biggest gain: {biggest_gain} ({days[gain_idx]} -> {days[gain_idx+1]})")
print(f"Volatility: {volatility}")
```

Output:

```
Max: 4055 (Sun) Min: 3845 (Mon)
Average: 3949.29
Biggest gain: 77 (Wed -> Thu)
Volatility: 286
```

Q42. 20-integer list: unique set, values appearing >1 time, values appearing exactly once (no Counter).

```
nums = [4, 8, 2, 4, 9, 8, 1, 7, 4, 3, 2, 6, 8, 5, 9, 1, 0, 3, 7, 2]

counts = {}
for n in nums:
    counts[n] = counts.get(n, 0) + 1

unique_values = set(nums)
more_than_once = sorted(n for n in unique_values if counts[n] > 1)
exactly_once = sorted(n for n in unique_values if counts[n] == 1)

print("Unique:", unique_values)
```

```
print("More than once:", more_than_once)
print("Exactly once:", exactly_once)
```

Output:

```
Unique: {0,1,2,3,4,5,6,7,8,9}
More than once: [1, 2, 3, 4, 7, 8, 9]
Exactly once: [0, 5, 6]
```

Q43. Smart Palindrome Checker (ignore case, spaces, punctuation).

```
def is_clean_palindrome(sentence):
    cleaned = ''.join(ch.lower() for ch in sentence if ch.isalnum())
    return cleaned == cleaned[::-1]

print(is_clean_palindrome("A man, a plan, a canal: Panama"))
print(is_clean_palindrome("Race car"))
print(is_clean_palindrome("No 'x' in Nixon"))
```

Output:

```
True
True
True
```

Q44. Manual Title Case (no .title()/.capitalize()).

```
def manual_title(sentence):
    result = ""
    capitalize_next = True
    for ch in sentence:
        if ch == ' ':
            result += ch
            capitalize_next = True
        else:
            result += ch.upper() if capitalize_next else ch.lower()
            capitalize_next = False
    return result

print(manual_title("hello world from bengaluru"))
```

Output:

```
Hello World From Bengaluru
```

Q45. Caesar Cipher Encryptor (wraps Z->A, preserves case and non-letters).

```
def caesar_encrypt(message, shift):
    result = ""
    for ch in message:
        if ch.isupper():
            result += chr((ord(ch) - ord('A') + shift) % 26 + ord('A'))
        elif ch.islower():
            result += chr((ord(ch) - ord('a') + shift) % 26 + ord('a'))
        else:
            result += ch
    return result

print(caesar_encrypt("Hello, India!", 3))
```

Output:

```
Khoor, Lqgld!
```

Q46. calculate_gst(amount, gst_rate=18, gst_inclusive=False) -> (base, gst, total).

```
def calculate_gst(amount, gst_rate=18, gst_inclusive=False):
    if not gst_inclusive:
        base = amount
        gst = amount * gst_rate / 100
        total = base + gst
    else:
        base = amount / (1 + gst_rate / 100)
        gst = amount - base
        total = amount
    return round(base, 2), round(gst, 2), round(total, 2)

print(calculate_gst(1000))
print(calculate_gst(1180, gst_inclusive=True))
print(calculate_gst(500, gst_rate=5))
```

Output:

```
(1000.0, 180.0, 1180.0)
(1000.0, 180.0, 1180.0)
(500.0, 25.0, 525.0)
```

Q47. Recursive power(base, exp) without ** or pow().

```
def power(base, exp):
    if exp == 0:
        return 1
    if exp < 0:
        return 1 / power(base, -exp)
    return base * power(base, exp - 1)

print(power(2, 10))
print(power(5, 0))
print(power(3, -2))
```

Output:

```
1024
1
0.1111111111111111
```

Q48. Identity Matrix Manipulations.

```
import numpy as np

identity = np.eye(5)
print("Identity:\n", identity)

scaled = identity * 10
print("x10:\n", scaled)

transposed = scaled.T
# Transpose of a diagonal matrix is itself: only off-diagonal elements
# would move on transpose, and all of them are already 0 here.
print("Transpose (same as original):\n", transposed)

print("Diagonal sum:", np.trace(scaled))
print("Total sum:", np.sum(scaled))
```

Output:

Diagonal sum: 50.0
Total sum: 50.0

Q49. Zomato Weekly Sales Analytics (no loops).

```
import numpy as np

sales = np.array([12000, 18500, 14200, 22100, 19800, 35200, 28400])
days = np.array(["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"])

total = np.sum(sales)
avg = np.mean(sales)
max_day = days[np.argmax(sales)]
above_avg_days = np.sum(sales > avg)
growth_pct = (sales[-1] - sales[0]) / sales[0] * 100
day_on_day_pct = np.diff(sales) / sales[:-1] * 100

print("Total:", total, "Average:", avg)
print("Max day:", max_day, "Days above avg:", above_avg_days)
print("Mon->Sun growth %:", round(growth_pct, 2))
print("Day-on-day % changes:", np.round(day_on_day_pct, 2))
```

Output:

Total: 150200 Average: 21457.14
Max day: Sat Days above avg: 3
Mon->Sun growth %: 136.67
Day-on-day % changes: [54.17 -23.24 55.63 -10.41 77.78 -19.32]

Q50. Border Pattern Challenge — 5x5 array, border 1 / interior 0, NO loops.

```
import numpy as np

border = np.ones((5, 5), dtype=int)
border[1:-1, 1:-1] = 0
print(border)
```

Output:

```
[[1 1 1 1 1]
 [1 0 0 0 1]
 [1 0 0 0 1]
 [1 0 0 0 1]
 [1 1 1 1 1]]
```